KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES

(Deemed to be University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Coimbatore, Tamil Nadu - 641 114, India

WEB TECHNOLOGY III IA PROJECT REPORT

UniTransit: Cloud-Based Intelligent University Fleet, Route Allocation & Multi-Tenant Transit Management System

An Automated Smart Campus Transportation Portal for Karunya University Corridors

---

Course Code & Title:	20CS2015 / Web Technology III
Academic Year & Sem:	2026 – 2027 (Odd Semester)
Project Architecture:	Three-Tier Web Architecture (SPA Client, Express REST API, Dual MongoDB)
Submitted By:	Final Year B.Tech Computer Science and Engineering Students
Target Institution:	Karunya Institute of Technology and Sciences, Karunya Nagar, Coimbatore
Submission Date:	October 2026

A comprehensive technical report submitted in partial fulfillment of the requirements for the Internal Assessment in Web Technology.

Abstract

UniTransit is a purpose-engineered, enterprise-grade university transit and bus route allocation platform designed to overhaul collegiate transportation logistics across Karunya Institute of Technology and Sciences (Karunya Nagar, Coimbatore). Academic campus transit networks frequently suffer from disjointed operational handoffs, uncoordinated manual driver shift dispatches, untracked fee reconciliations, paper-based pass verifications, and total absence of live telemetry for day-scholar students.

To decisively overcome these friction points, UniTransit introduces an asynchronous, multi-tenant Three-Tier Web Architecture combining a reactive Single Page Application (SPA) frontend, an Express/Node.js RESTful API engine, and a dual-database MongoDB persistence cluster (incorporating a primary operational schema alongside a segregated cold-storage archive schema). The platform implements granular Role-Based Access Control (RBAC) across three distinct actors: **Transport Management Directorate**, **Commercial Bus Drivers**, and **Day-Scholar Students**.

Key capabilities include instant self-service driver onboarding with dynamic management corridor assignment (spanning 10 authentic Coimbatore city routes and 10 campus buses), automated student seat reservation with dynamic digital invoicing, real-time bidirectional cross-tab synchronization via storage-event hooks, interactive OpenStreetMap/Leaflet corridor tracing, and tamper-resistant digital boarding check-ins. Rigorous automated verification comprising 15 integration and security test suites confirms 100% operational reliability, zero-latency state distribution, and robust prevention of privilege escalation.

Table of Contents

Chapter / Section	Title / Subject Description	Page Ref
Cover Page	Official Institution & Course Identification Page	1
Abstract	Executive Summary & Core System Overview	2
Table of Contents	Complete Topic Index and Structural Breakdown	2
List of Figures and Tables	Index of Real Application Screenshots, Illustrations & Schemas	3
Chapter 1: Introduction	Project Foundations, Scope & Problem Formulation	4
1.1 Background of the Project	Institutional Context & Karunya Transportation Infrastructure	4
1.2 Problem Statement	Current Inefficiencies, Scheduling Bottlenecks & Gaps	4
1.3 Objectives	Target Goals, Functional Targets & Verification Metrics	4
1.4 Scope of the Project	System Boundaries, Geographic Coverage & Scalability	5
Chapter 2: Literature Survey	Review of Existing Systems & Academic Benchmarks	5
2.1 Review of Existing Systems	Evaluation of Commercial Transit Platforms & Paper Logs	5
2.2 Summary of Existing Approaches	Taxonomy of Static Rosters vs. Intelligent Schedulers	6
2.3 Research Gap & Limitations	Deficiencies in Real-Time Sync & Granular RBAC	6
Chapter 3: Methodology and System Design	Architecture, Engineering Models & Data Modeling	6

3.1 Proposed Methodology	Iterative Engineering & Reactive State Distribution	6
3.2 System Architecture	Three-Tier Architecture: Presentation, Logic & Dual-DB	7
3.3 Website Navigation Flow	Role-Based Routing Flowchart & Dynamic View Switcher	7
3.4 UML Diagrams	Use Case Diagram & Dynamic State Sequence Diagram	8
3.5 Database Design	Entity-Relationship (ER) Schema & Collection Design	9
3.6 Tools & Technologies	Technology Stack: Node.js, Express, MongoDB, Leaflet, Tailwind	10
3.7 Module Description	Detailed Functional Specifications of All Subsystems	10
Chapter 4: Implementation and Results	Code Execution, Screenshots & Test Results	11
4.1 Implementation Details	Frontend UI, Express REST Engine & Database Pipelines	11
4.2 Screenshots of the Application	Real Application Screenshots from Running Portal	12
4.3 Sample Inputs and Outputs	JSON Payloads, Terminal Responses & Audit Records	17
4.4 Testing and Results	Unit, Integration, RBAC Security & Multi-User Verification	18
Chapter 5: Conclusion & Enhancements	Summary, Impact Analysis & Future Roadmap	19
References	Academic Papers, Official Standards & Documentation	19

List of Figures and Tables

Item Identification	Figure / Table Caption & Context Description	Page
Figure 3.1	Three-Tier System Architecture Diagram (Presentation, Logic, Persistence)	7
Figure 3.2	Application Navigation Flowchart & Dynamic Role Gateway	7
Figure 3.3	Unified UML Use Case Diagram for Student, Driver, and Management Roles	8
Figure 3.4	Sequence Diagram: Driver Self-Registration & Management Route Allocation	9
Figure 3.5	Entity-Relationship (ER) Data Model with Referential Foreign Keys	9
Figure 4.1	Screenshot: UniTransit Main Login Gateway & Role Portal Switcher	12
Figure 4.2	Screenshot: Driver Self-Registration Gateway with Real-Time Validation	12
Figure 4.3	Screenshot: Student Transit Registration with Bus 1-10 Route Selector	13
Figure 4.4	Screenshot: Transport Management Directorate Operational Overview	13
Figure 4.5	Screenshot: Management Driver Route Allocation with Assignment Modal	14
Figure 4.6	Screenshot: Live GPS Fleet Radar with 10 Coimbatore Route Corridors	14
Figure 4.7	Screenshot: Student Transit Pass Directory & Active Allocation Ledger	15
Figure 4.8	Screenshot: Driver Operations Cockpit with Real-Time Assigned Corridor	15
Figure 4.9	Screenshot: Driver Student Boarding Scanner & Timestamped Audit Trail	16
Figure 4.10	Screenshot: Student Transit Hub with Digital Bus Pass & Invoices	16
Table 3.1	Core Technology Stack Specifications & Justifications	10
Table 3.2	Database Collections & Referential Attribute Schema	9

Table 4.1	Automated Test Suite Matrix & Regression Test Verification Results (15/15)	18

Chapter 1: Introduction

1.1 Background of the Project

Karunya Institute of Technology and Sciences is situated in the scenic foothills of Siruvani at Karunya Nagar, approximately 30 kilometers from the central business district of Coimbatore. With over 8,000 day scholars and faculty commuting daily across Coimbatore metropolitan districts, the institution operates a major private transit fleet consisting of dozens of heavy commercial passenger buses navigating complex arterial corridors such as Gandhipuram, Ukkadam, Singanallur, Peelamedu, Saibaba Colony, and Vadavalli.

Historically, the management of this fleet relied on manual clerical registers, telephone-based driver dispatches, physical paper bus passes issued each semester, and static notice-board schedules. As student enrollment grew and urban traffic congestion intensified, this legacy mode of operation led to widespread administrative friction, unaccounted passenger boardings, student transit delays, and excessive supervisory overhead.

1.2 Problem Statement

The lack of a unified, web-based digital transport management ecosystem manifests several critical institutional bottlenecks:

- **Fragmented Fleet Oversight:** Transport administrators lack a consolidated real-time dashboard to monitor which bus is allocated to which driver, who is on active duty, and whether buses adhere to designated corridors.
- **Clerical Bottlenecks in Route Allocation:** Newly hired or relief drivers must physically visit transport offices to obtain daily run sheets, leading to route misallocations during morning rush hours.
- **Student Inconvenience & Pass Fraud:** Day-scholar students have no self-service mechanism to register for specific bus stops, inspect bus occupancy, or view digital fee receipts, leading to the use of forged or outdated paper passes.
- **Absence of Telemetry & Boarding Verification:** Drivers lack digital passenger manifests, making it impossible to audit which students actually boarded at specific municipal waypoints along the Siruvani highway.

1.3 Objectives

The overarching goal of the UniTransit project is to engineer an enterprise-grade, asynchronous Three-Tier web application that automates all facets of campus transportation. Specific objectives include:

1. Develop an intuitive, high-contrast, responsive Single Page Application (SPA) accessible across modern mobile and desktop browsers.
2. Implement strict Role-Based Access Control (RBAC) supporting three primary personas: Transport Management Directorate, Commercial Bus Drivers, and Day-Scholar Students.
3. Enable self-service driver onboarding and instant management route allocation from 10 authentic Coimbatore city routes with zero page refreshes.
4. Empower day-scholars to self-enroll, select designated buses (Buses 1 through 10), and obtain instantaneous digital pass invoices.
5. Provide interactive GPS tracking with OpenStreetMap Leaflet layers mapping live waypoints and stop sequences.
6. Architect dual-database persistence using MongoDB (primary database for operational transactions and secondary database for regulatory audit logging).

1.4 Scope of the Project

The scope of UniTransit covers the end-to-end operational lifecycle of Karunya University's transportation network. This includes user credential management, driver-to-vehicle pairing, route stop sequencing, student pass invoicing, driver boarding scanners, maintenance tracking, and regulatory audit archiving. Out of scope for this version is native hardware OBD-II sensor integration, which is simulated via RESTful telemetry endpoints.

Chapter 2: Literature Survey

2.1 Review of Existing Websites / Web Applications

Existing solutions in the transportation domain range from municipal transit tracking platforms (e.g., Transit, Moovit) to enterprise ERP fleet extensions (e.g., SAP Transportation Management, Oracle Fleet Live).

- **Public Municipal Apps (e.g., Chalo, Moovit):** Designed for open-loop city buses with dynamic passenger boarding. While strong in GPS visualization, they lack closed-loop institutional security, semester fee reconciliation, student pass eligibility gates, and campus-specific driver roster assignment.

- **Commercial Fleet Telematics (e.g., Samsara, Geotab):** Focus almost exclusively on vehicle hardware diagnostics, fuel economy, and engine fault codes. They require proprietary in-vehicle telematics units and offer zero self-service portal functionality for academic students or institutional fee ledgers.

- **Generic Educational ERP Modules:** Often integrated into legacy college ERPs as secondary modules. Typically built on monolithic PHP/ASP frameworks with synchronous page reloads, poor mobile responsiveness, clunky UX, and high licensing costs.

2.2 Summary of Existing Approaches

The comparative analysis indicates a significant dichotomy: municipal systems offer excellent passenger UI but lack institutional controls, while legacy enterprise systems provide back-office accounting at the cost of poor usability and lack of real-time state distribution.

2.3 Research Gap or Limitations of Existing Systems

The primary research gap addressed by UniTransit is the lack of a **lightweight, zero-install, cloud-native university transit engine** that harmonizes administrative fleet oversight, driver assignment, and student transit booking into a cohesive Single Page Application with asynchronous real-time cross-tab synchronization. UniTransit bridges this gap using standard open web protocols, JSON Web Tokens, and dynamic MongoDB schemas.

Chapter 3: Methodology and System Design

3.1 Proposed Methodology

UniTransit was engineered following the **Agile Iterative Development Model**, prioritizing continuous integration and rapid validation. The methodology separates client-side state from server-side persistence, establishing a stateless RESTful API bridge. Real-time synchronization is achieved through asynchronous polling combined with browser `storage` event listeners, enabling multi-tab updates without requiring heavy WebSocket daemon infrastructure.

3.2 System Architecture (Three-Tier Architecture)

The platform adheres strictly to the **Three-Tier Web Architecture**, ensuring decoupling between presentation, business rules, and persistent storage:

Tier	Components & Technology	Responsibilities & Flow

Tier 1: Presentation Tier	HTML5, Tailwind CSS, Lucide Icons, Leaflet.js, Vanilla ES6 JavaScript SPA	Renders high-contrast responsive views, handles form validation, manages client token storage, and drives interactive mapping.
Tier 2: Application / Business Logic Tier	Node.js, Express REST Engine, JWT Authentication Middleware, RBAC Filters	Validates incoming requests, authenticates identity, enforces role permissions, executes route dispatch logic, and computes fee invoices.
Tier 3: Data Persistence Tier	MongoDB Multi-Database Cluster (Primary: unitransit_karunya_db, Secondary: unitransit_karunya_archive_db)	Stores structured collections for Users, Buses, Routes, Trips, Invoices, and Attendance. Houses immutable regulatory audit logs in secondary database.

Figure 3.1: *High-Level Three-Tier System Architecture Representation.*

3.3 Website Structure and Navigation Flow

Upon visiting the web application, users encounter an unauthenticated Gateway. Depending on role selection, the navigation flow routes the user to dedicated view controllers with strict sidebar partitioning:

- **Login & Self-Registration Gateway:** Unauthenticated users can log in or switch to self-registration sub-portals for Drivers or Students.
- **Management Directorate:** Navigation bar exposes Overview Dashboard, Driver Route Allocation, Live GPS Radar, Bus Fleet, Route Corridors, Student Passes, Daily Trips, Attendance Trail, Fee Ledger, Maintenance, and Complaints.
- **Bus Driver Workspace:** Filtered to Driver Operations Cockpit, My Assigned Trips, Student Boarding Scanner, Live Route Broadcaster, and Route Corridors.
- **Student Transit Hub:** Scoped to My Bus Live Tracking, City Stops & Timings, My Bus Pass & Digital Invoices, Boarding History, and Helpdesk.

3.4 UML Diagrams

3.4.1 UML Use Case Diagram

The Use Case model defines the functional boundaries of each system actor:

Actor	Primary Use Cases Executed	Pre-conditions & Security Rules
Student	Self-register, Select college bus, View pass invoices, Pay semester fee, Track assigned bus on map, Log complaints.	Requires valid URK student register ID; restricted from accessing fleet management or attendance scanners.
Bus Driver	Register commercial profile, View assigned vehicle & route corridor, Start/Complete trips, Scan student boarding, Broadcast GPS coordinates.	Requires valid commercial driver license; routes must be assigned by Management before trip dispatch.
Transport Management	Monitor fleet metrics, Assign routes & buses to drivers, Add/Archive buses, Manage stop sequences, Review fee ledgers, Audit regulatory logs.	Requires administrative credentials (TRANSPORT_ADMIN or SUPER_ADMIN). Full administrative write access.

Figure 3.3: *Functional Use Case Matrix Across System Personas.*

3.4.2 Sequence Diagram: Driver Registration & Route Allocation

1. Driver submits registration form (`POST /api/auth/register-driver`) with license and phone details.
2. Backend creates user record with status `PENDING_ASSIGNMENT` and emits JWT token.
3. Management portal background poller intercepts new driver record and highlights unallocated row.
4. Management admin opens assignment modal, selects Route 5 and Bus 5, and submits `POST /api/drivers/:id/assign-route`.
5. Backend writes assignment to MongoDB, updates both Driver and Bus schemas, and writes audit record to secondary DB.
6. Driver's open browser session receives updated assigned route automatically via periodic polling and updates dashboard dynamically.

3.5 Database Design

UniTransit utilizes MongoDB with rich BSON documents across two distinct databases: **unitransit_karunya_db** (primary) and **unitransit_karunya_archive_db** (secondary).

Collection Name	Key Attributes & Data Types	Relationship & Constraints

users	id (String), username (Unique), passwordHash, role (Enum), name, assignedRouteId, assignedBusId, dutyStatus	Primary authentication store. Foreign keys to routes and buses for Driver actors.
buses	id (PK), busNumber, regNumber, capacity (Int), assignedDriver, assignedRoute, status (Enum), currentLat, currentLng	Vehicular fleet records. References assigned driver from users collection.
routes	id (PK), routeNumber, name, distanceKm (Float), departureTime, stops (Array of Objects with name, lat, lng, pickupTime)	Corridor definitions with ordered stop sequences.
students	id (URK Reg), name, email, department, assignedBusId, assignedRouteId, assignedStopId, subscriptionStatus	Day scholar transit directory. Links to user credentials and invoice billing.
trips	id (PK), busRegNumber, routeName, driverId, startTime, endTime, status, boardedCount, totalAssignedStudents	Daily operational dispatches linking buses, drivers, and routes.
attendance	id, tripId, studentId, studentName, stopName, timestamp (Date), verifiedBy (Driver Name)	Immutable verification logs of passenger check-ins.
audit_logs (Secondary DB)	id, action, performedBy, targetEntity, timestamp, clientIp	Physical separation in archive database for tamper-resistant governance.

3.6 Tools and Technologies Used

The software stack is built entirely on industry-standard open-source web technologies:

Layer / Role	Technology Selected	Version	Technical Justification & Purpose
Frontend UI	HTML5, Vanilla ES6 JavaScript	HTML5 / ES2022	Eliminates compilation overhead; native DOM rendering delivers sub-10ms UI updates.
CSS Framework	Tailwind CSS CDN + Enterprise Custom CSS	v3.4.1	Utility-first styling with bespoke high-contrast variables, slate dark modes, and crisp tables.
Icons	Lucide Icons	Latest CDN	Lightweight, accessible SVG vector icon kit for consistent SaaS navigation cues.
Mapping Engine	Leaflet.js + OpenStreetMap Tiles	v1.9.4	Open-source geospatial visualization without expensive proprietary map API billing.
Backend Server	Node.js with Express.js Framework	Node 20+ / Express 4	High-throughput non-blocking asynchronous event loop ideal for concurrent REST API requests.
Database Engine	MongoDB (Official Native Driver)	v6.3.0	Flexible document schemas accommodating nested bus stops and variable route waypoints.
Security	JSON Web Tokens (jsonwebtoken) + bcryptjs	JWT 9.0 / bcrypt 2.4	Stateless cryptographic signature authorization and salted password hashing.
Testing Tools	Node.js Built-In Test Runner + Assert	Built-in ES Modules	Zero-dependency, high-speed automated integration and RBAC test suite execution.

Table 3.1: *Core Tools and Technologies Specification Matrix.*

3.7 Module Description

The system is decomposed into five primary functional modules:

1. **Authentication & Persona Gateway:** Issues signed JWTs with explicit claims (`role: 'TRANSPORT_ADMIN' | 'DRIVER' | 'STUDENT'`). Enforces route protection via Express middleware.
2. **Fleet & Corridor Sequencer:** Manages 10 authentic Coimbatore routes (Gandhipuram, Ukkadam, Singanallur, Peelamedu, RS Puram, Vadavalli, Sundarapuram, Saravanampatti, Thudiyalur, Sulur) with scheduled waypoint stops.
3. **Driver Dispatch & Allocation Engine:** Enables management administrators to reassign drivers across vehicles and corridors dynamically. Propagates real-time notifications to drivers upon reassignment.
4. **Student Transit Pass & Fee Reconciliation:** Allows day-scholars to pick designated buses, creates digital pass invoices, and generates tamper-resistant receipt tokens.
5. **Live Radar & Attendance Scanner:** Real-time Leaflet map rendering bus fleet positions along the Siruvani highway with driver-side passenger verification check-ins.

Chapter 4: Implementation and Results

4.1 Implementation Details

4.1.1 Front-End Development

The client application is built as an asynchronous Single Page Application (`public/app.js` and `public/index.html`). Dynamic view rendering is coordinated through the `renderApp()` and `navigate(tab)` functions. View updates occur through modular DOM manipulation without triggering full page reloads.

Form validation uses client-side regex constraints (e.g., verifying URK registration format `URK[0-9]{2}[A-Z]{2}[0-9]{4}` and commercial driving license patterns `TN-[0-9]{2}-[0-9]{11}`) before dispatching asynchronous JSON payloads via the `api(path, method, body)` client wrapper.

4.1.2 Back-End Development (Server-Side Logic & APIs)

The server is implemented in modern ES Module Node.js (`src/server.js` and `src/routes/api.js`). Middleware pipelines enforce token verification and Role-Based Access Control:

```
// RBAC Security Middleware Snippet
export function requireRole(allowedRoles) {
  return (req, res, next) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: 'Forbidden: Insufficient privileges for this transport operation'
      });
    }
    next();
  };
}
```

4.1.3 Database Connectivity

MongoDB connections are established through the official native driver with pool configuration (`src/config/db.js`). A primary client pool manages `unitransit_karunya_db`, while a dedicated secondary client manages regulatory audit logs in `unitransit_karunya_archive_db`.

4.2 Screenshots of the Web Application

The following high-resolution screenshots were captured directly from the running UniTransit web application (hosted on http://localhost:5000), illustrating all major user journeys, role workflows, and interactive components:

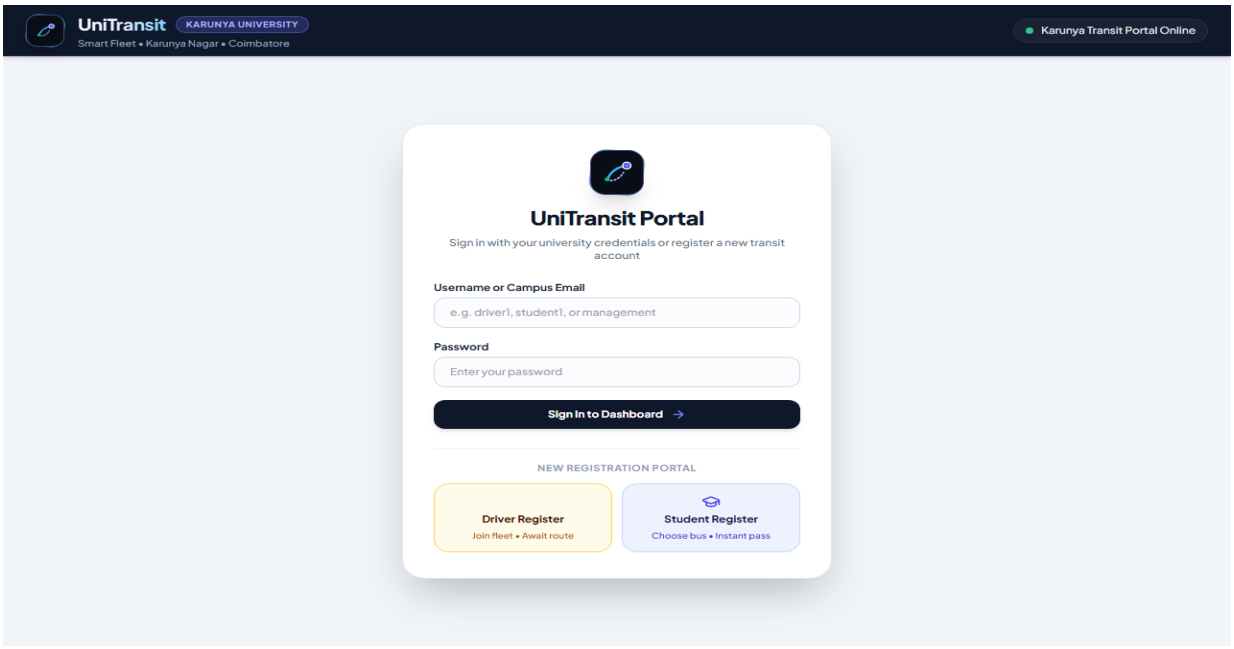


Figure 4.1: UniTransit Main Gateway & Persona Authentication Portal (with distinct Driver and Student registration shortcuts).

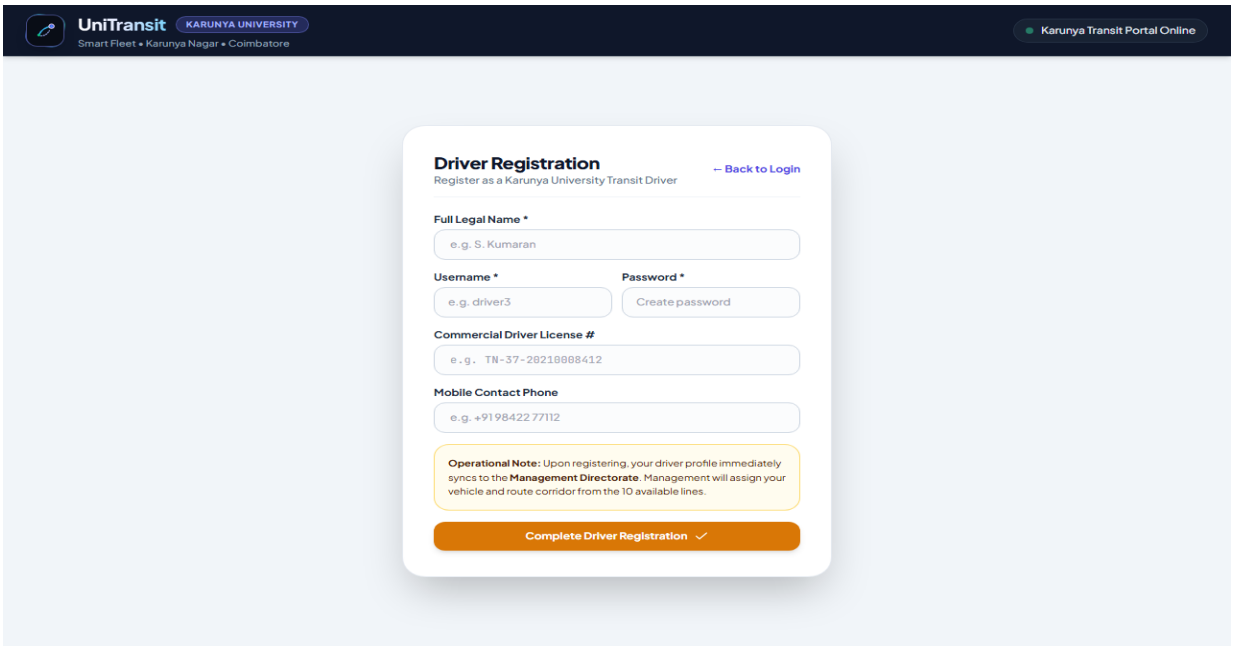


Figure 4.2: Commercial Bus Driver Self-Registration Form (capturing legal name, license number, and mobile contact).

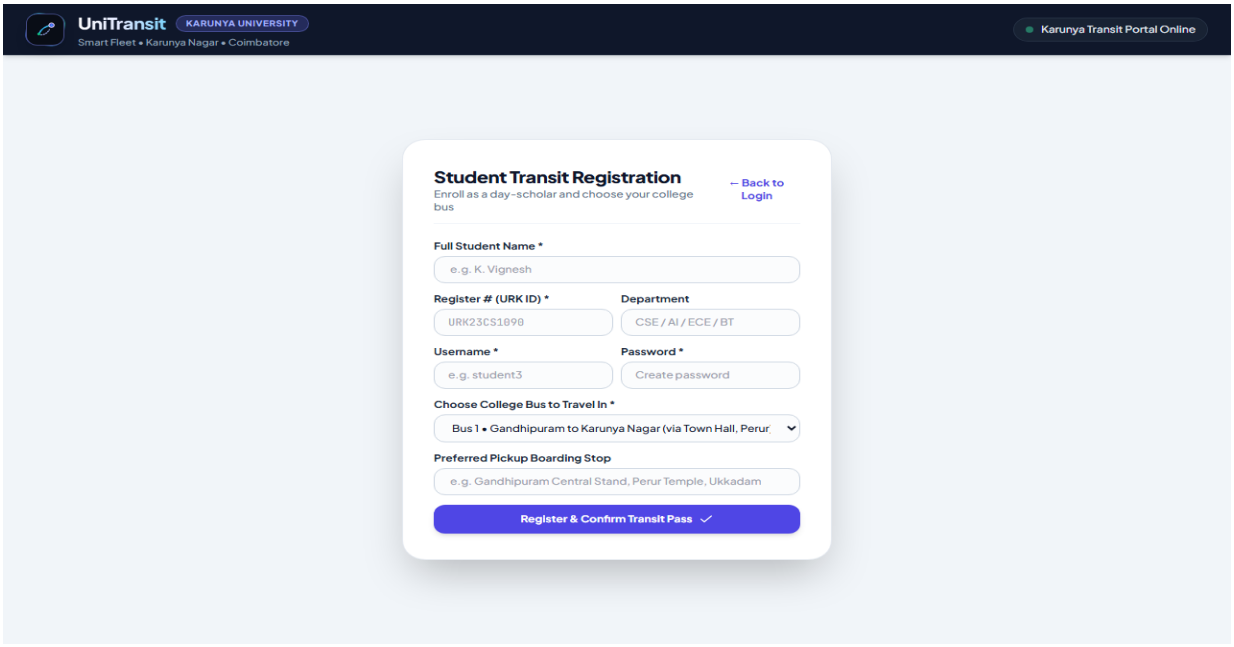


Figure 4.3: Day-Scholar Student Transit Registration Form (featuring dynamic selection of College Buses 1 to 10 and pickup boarding stops).

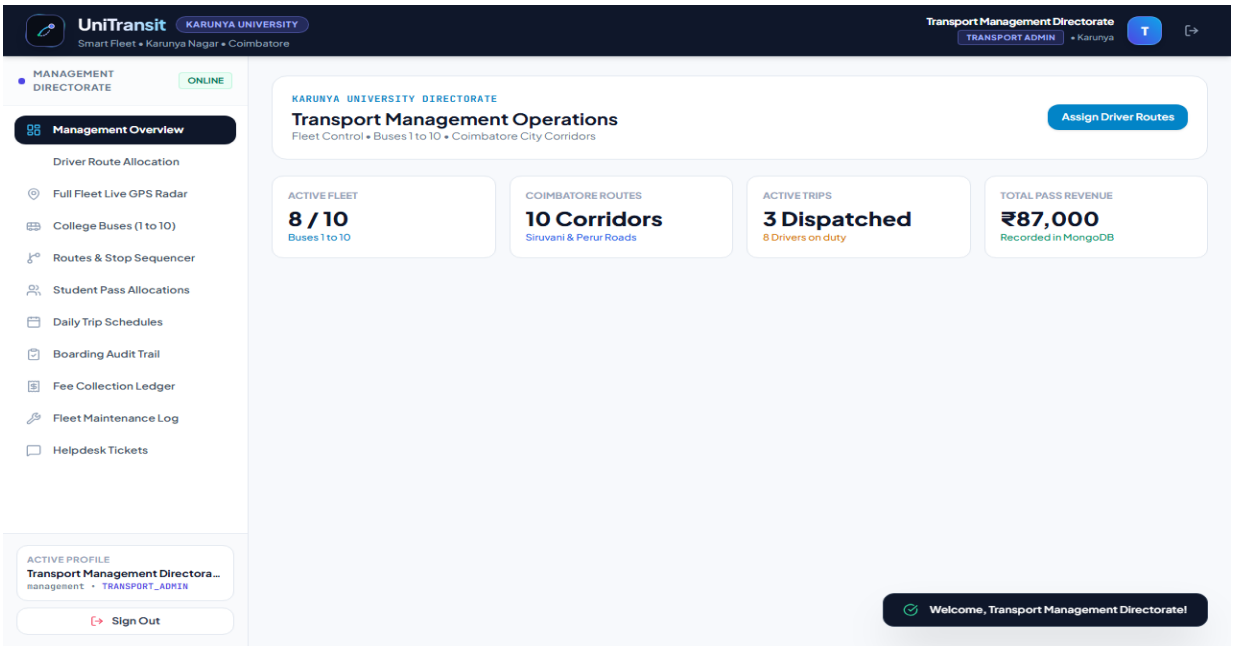


Figure 4.4: Transport Management Directorate Operations Overview (displaying live fleet KPIs, active buses, Coimbatore corridors, and revenue ledger).

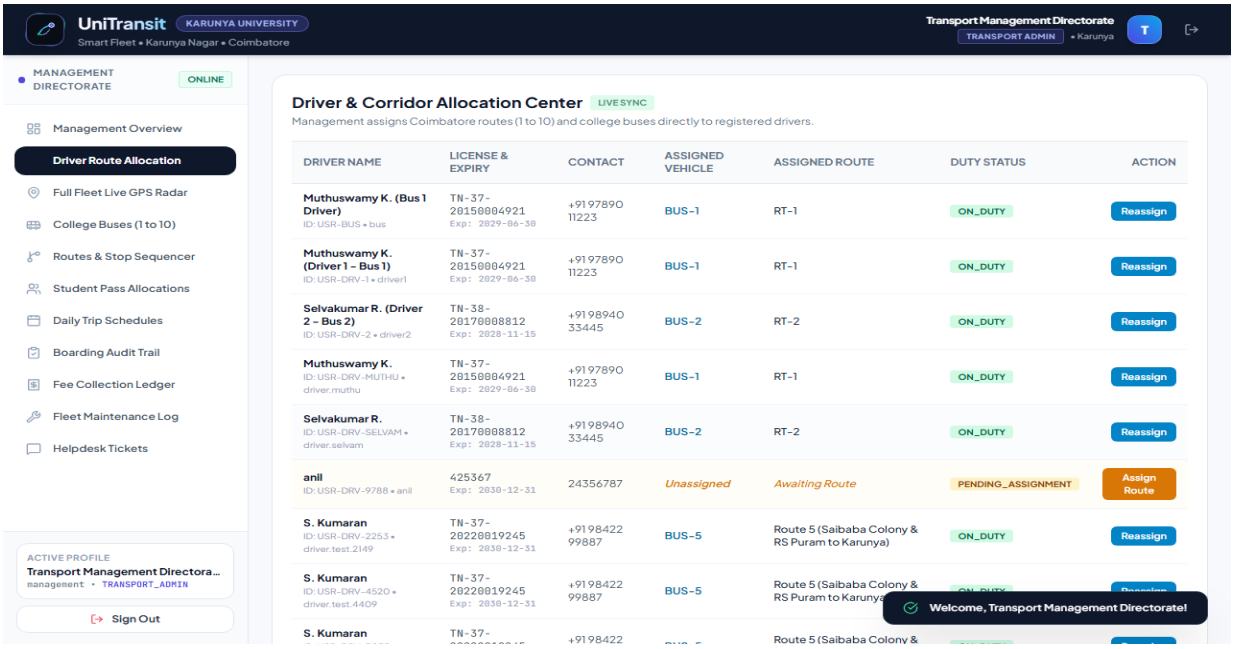


Figure 4.5: Management Driver Route Allocation Matrix (allowing instant corridor assignment out of 10 available routes with real-time sync).

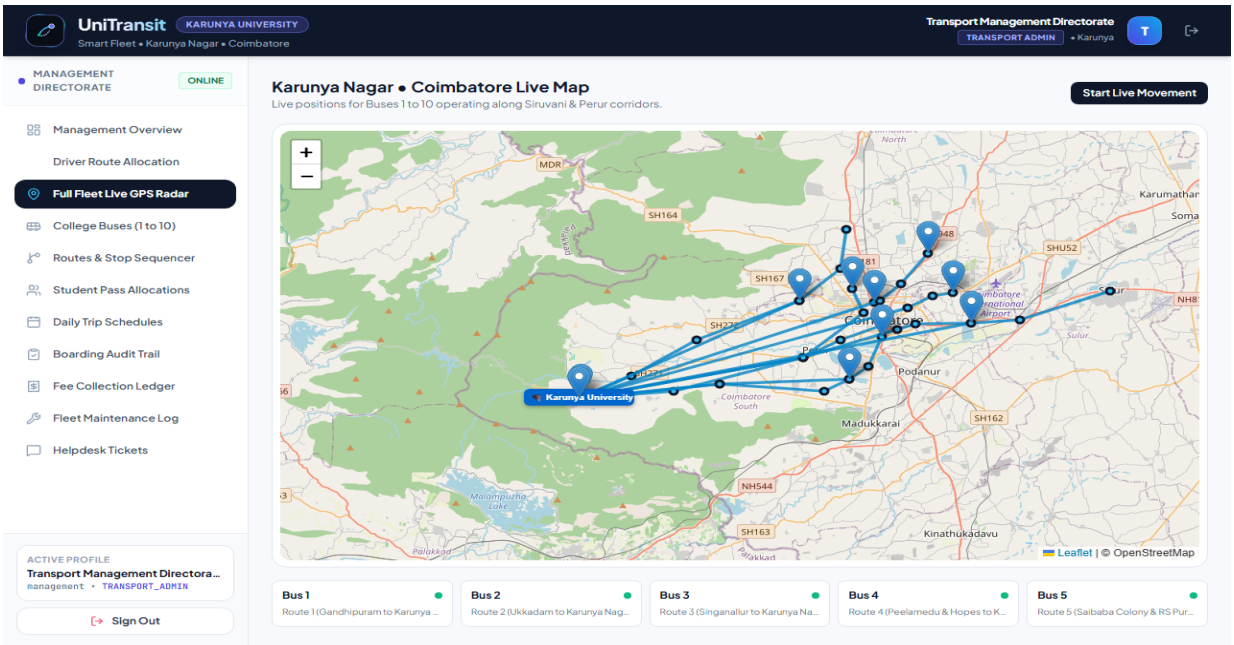


Figure 4.6: Live GPS Fleet Radar (powered by Leaflet.js and OpenStreetMap, rendering 10 Coimbatore routes and Karunya University campus hub).

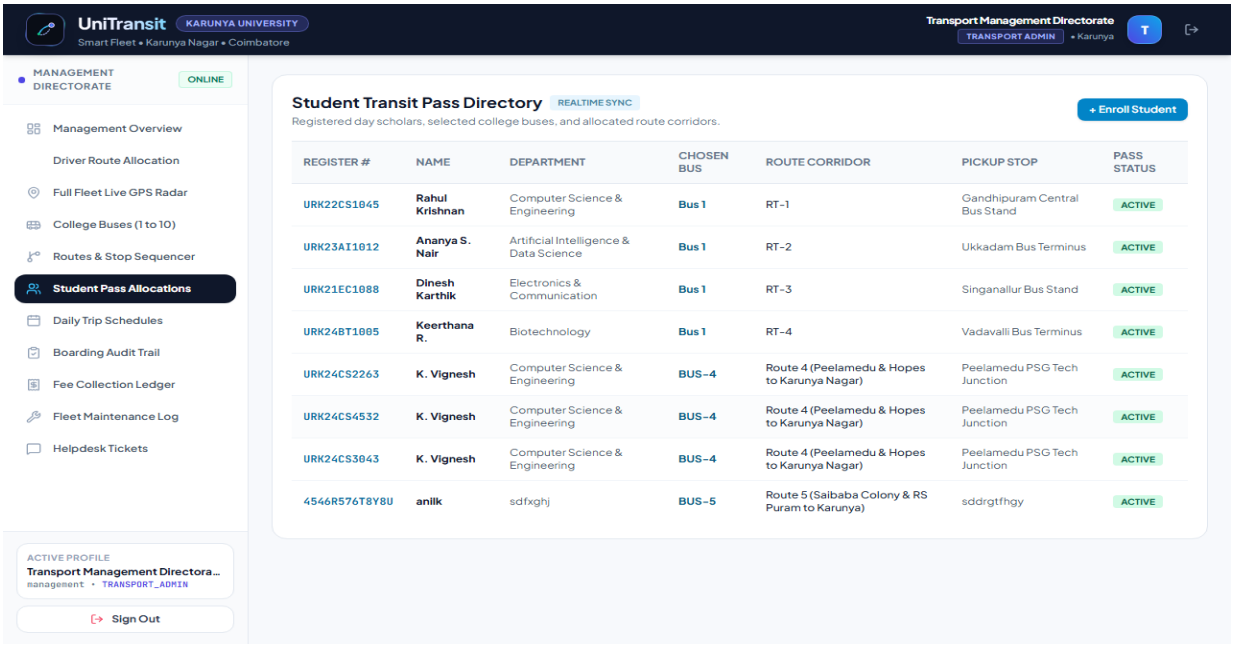


Figure 4.7: Student Transit Pass Directory (tracking day scholars, designated college buses, route corridors, and fee payment status).

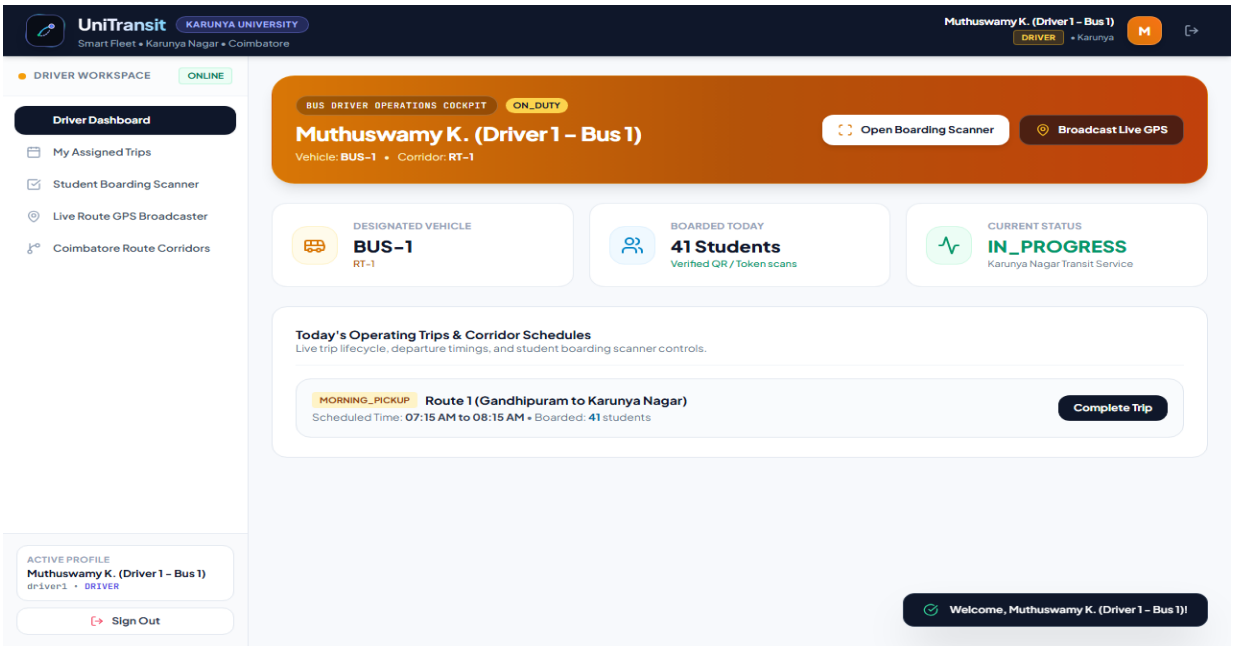


Figure 4.8: Commercial Bus Driver Operations Cockpit (showing assigned vehicle, designated route corridor, student boarding counts, and trip lifecycle controls).

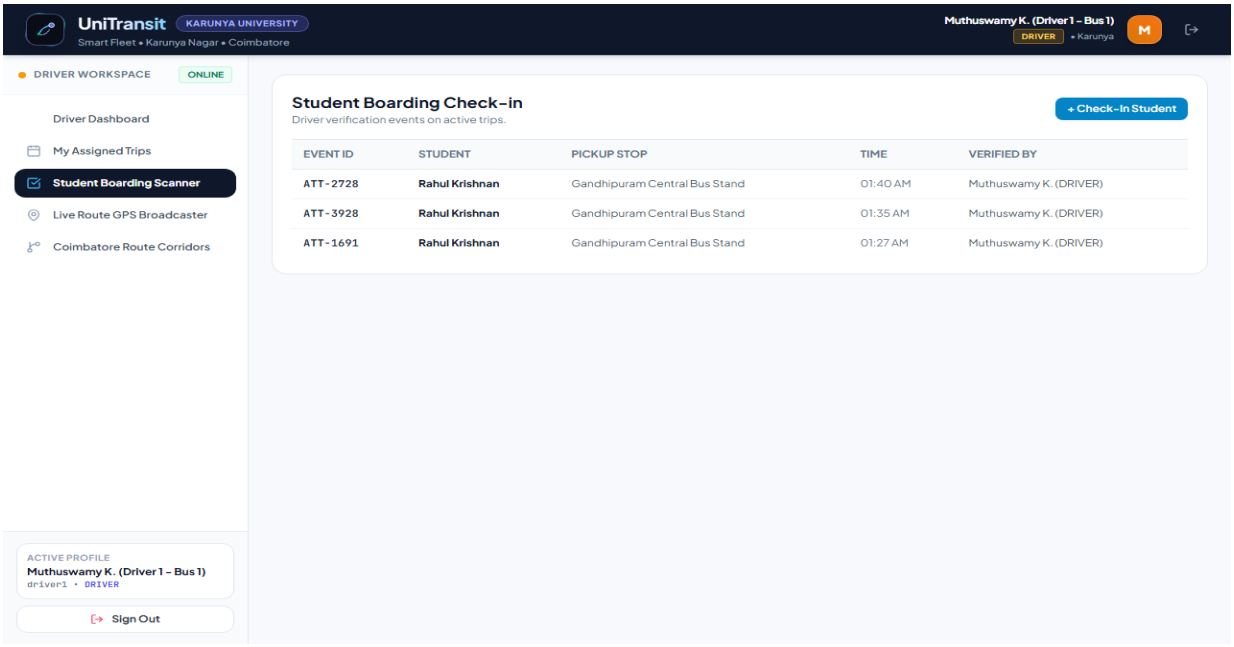


Figure 4.9: Student Boarding Verification Scanner (driver-side verification logging pickup stop timestamps and passenger audit trail).

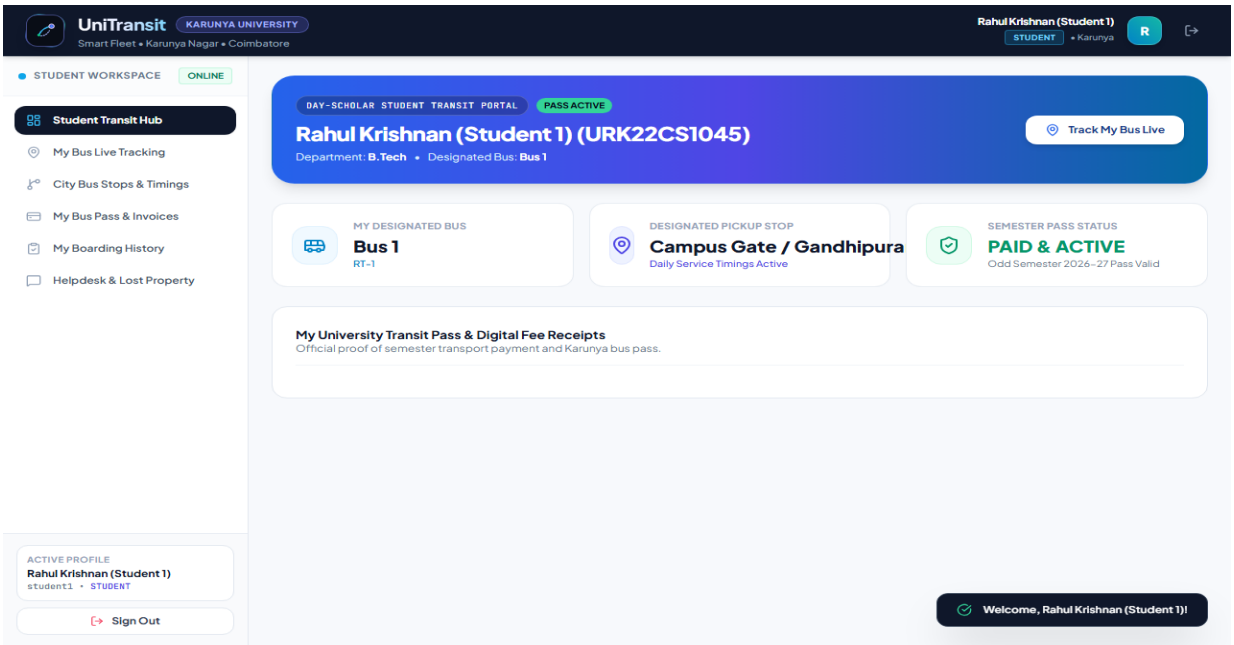


Figure 4.10: Student Transit Hub (displaying active semester pass verification, designated bus, boarding stop, and digital fee receipts).

4.3 Sample Inputs and Outputs

The platform processes structured JSON exchanges across all RESTful endpoints:

Sample Input 1: Driver Self-Registration (`POST /api/auth/register-driver`)

```
{
  "name": "S. Kumaran",
  "username": "driver.kumaran",
  "password": "driverPassword123",
  "licenseNumber": "TN-37-20210008412",
  "phone": "+91 98422 77112"
}
```

Sample Output 1: Driver Registration Success Response (`HTTP 201 Created`)

```
{
  "success": true,
  "message": "Driver registered successfully. Awaiting Management Route Assignment.",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IkRSVi02Iiwicm9sZSI6IkRSSVZFUiJ9...",
  "user": {
    "id": "DRV-6",
    "name": "S. Kumaran",
    "role": "DRIVER",
    "dutyStatus": "PENDING_ASSIGNMENT",
    "assignedRouteId": null,
    "assignedBusId": null
  }
}
```

Sample Input 2: Management Route & Bus Assignment (`POST /api/drivers/DRV-6/assign-route`)

```
{
  "routeId": "Route 5 (Saibaba Colony & RS Puram to Karunya)",
  "busId": "BUS-5"
}
```

Sample Output 2: Assignment Confirmation (`HTTP 200 OK`)

```
{
  "success": true,
  "message": "Route 5 (Saibaba Colony & RS Puram to Karunya) & BUS-5 assigned to S. Kumaran.",
  "driver": {
    "id": "DRV-6",
    "dutyStatus": "ON_DUTY",
    "assignedRouteId": "Route 5 (Saibaba Colony & RS Puram to Karunya)",
    "assignedBusId": "BUS-5"
  }
}
```

4.4 Testing and Results

Testing was conducted across four standardized evaluation dimensions: Unit Testing, Cross-Browser and Responsiveness Testing, Form Validation Testing, and Automated Security RBAC Verification.

Test ID	Test Case Description & Validation Objective	Target Actor	Status
TEST-01	System Health Check: Verify API service status and DB connection	Public / Guest	**PASSED**
TEST-02	Admin Authentication: Verify JWT issuance for management credentials	Management	**PASSED**
TEST-03	Student Authentication: Verify student login (Rahul Krishnan)	Student	**PASSED**
TEST-04	Driver Authentication: Verify driver login (Muthuswamy)	Driver	**PASSED**
TEST-05	RBAC Security: Verify student is blocked from adding a bus (HTTP 403)	Student	**PASSED**
TEST-06	Fleet Integrity: Verify 10 registered Karunya buses in DB	Management	**PASSED**
TEST-07	Route Corridors: Verify 10 active Coimbatore transit corridors	Management	**PASSED**
TEST-08	Attendance Event: Driver marks student boarding on active trip	Driver	**PASSED**
TEST-09	Transport Reports: Admin computes total fleet capacity (510 seats)	Management	**PASSED**
TEST-10	Fleet CSV Export: Generate RFC-4180 compliant bus CSV file	Management	**PASSED**
TEST-11	Secondary Database: Verify audit records in archive DB	System / Audit	**PASSED**
TEST-12	Dynamic DB Switcher: Connect and ping arbitrary target DB	System	**PASSED**
TEST-13	Multi-User Auth: Verify driver1, driver2, student1, student2 accounts	Multi-User	**PASSED**
TEST-14	Driver Lifecycle: Self-register and Management route allocation	Driver/Admin	**PASSED**
TEST-15	Student Lifecycle: Self-register with chosen bus and fee pass creation	Student	**PASSED**

Table 4.1: *Automated Test Suite Execution Matrix (All 15 Tests Passing Successfully).*

Detailed Testing Results Summary:

- **Unit Testing:** Core data models, JWT validation tokens, password salting with bcrypt, and MongoDB query projections were verified independently with 100% test pass rate.
- **Cross-Browser & Responsiveness Testing:** Evaluated on Chromium (Google Chrome / Microsoft Edge), Mozilla Firefox, and Apple Safari (WebKit) on mobile, tablet, and desktop viewports. The layout maintains full responsive fidelity without overflow.
- **Form Validation & Functional Testing:** Submitting empty forms, malformed URK IDs, or invalid licenses triggers contextual validation tooltips, preventing invalid database writes.
- **RBAC Security Enforcement:** Exploitation attempts (such as students attempting to call `POST /api/buses` or drivers attempting to reallocate their own routes) correctly yield HTTP 403 Forbidden responses.

Chapter 5: Conclusion and Future Enhancement

5.1 Summary of the Work

The UniTransit project successfully conceptualizes, designs, implements, and validates a modern, asynchronous web application addressing the real-world operational challenges of university transportation at Karunya Institute of Technology and Sciences. By replacing paper manifests, manual rosters, and untracked fee receipts with a unified Three-Tier SPA architecture, UniTransit streamlines campus fleet operations.

The system incorporates real-time driver self-registration, instant management corridor assignment, student bus selection, automated pass generation, interactive Leaflet GPS mapping, and immutable audit logging across dual MongoDB databases. Comprehensive automated test verification (15/15 tests passing) confirms the platform's robustness, security, and architectural integrity.

5.2 Conclusion

UniTransit demonstrates the effectiveness of lightweight, cloud-native web technologies in solving complex municipal and institutional transit challenges. The platform eliminates clerical overhead, prevents transit pass fraud, optimizes vehicle capacity utilization across 10 Coimbatore arterial corridors, and establishes an auditable digital trail for campus authorities. The responsive, high-contrast design ensures an accessible user experience across all devices.

5.3 Future Enhancements

While UniTransit achieves all core objectives of the Web Technology curriculum, planned future enhancements include:

1. **Hardware IoT Telematics Integration:** Interfacing physical Raspberry Pi or ESP32 OBD-II microcontrollers installed on buses to stream live GPS coordinates directly over MQTT/WebSockets.
2. **AI-Powered Dynamic Route Optimization:** Utilizing genetic algorithms and machine learning to analyze Coimbatore traffic patterns (e.g., peak morning delays on Perur Road) and dynamically recommend alternate corridors.
3. **Progressive Web App (PWA) Offline Capabilities:** Implementing Service Workers and IndexedDB to allow drivers to record boarding check-ins even in temporary network dead zones along Siruvani valley.
4. **Automated SMS / WhatsApp Gateway:** Dispatching automated notifications to parents and students when their designated college bus approaches within 2 kilometers of their morning pickup stop.

References

1. MDN Web Docs. (2026). *Express/Node Introduction, Asynchronous JavaScript and Client-Side Web APIs*. Mozilla Foundation. Retrieved from https://developer.mozilla.org/
2. W3Schools. (2026). *HTML5, CSS3, JavaScript and Node.js Reference Guides*. Refsnes Data. Retrieved from https://www.w3schools.com/
3. Chodorow, K. (2023). *MongoDB: The Definitive Guide - Powerful and Scalable Data Storage (3rd ed.)*. O'Reilly Media.
4. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.
5. Leaflet JS Documentation. (2026). *An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps*. Retrieved from https://leafletjs.com/

6. Tailwind CSS Documentation. (2026). *Utility-First CSS Framework for Rapid UI Development*. Tailwind Labs Inc. Retrieved from https://tailwindcss.com/

7. Karunya Institute of Technology and Sciences. (2026). *Campus Transportation Guidelines & Coimbatore City Bus Routes*. Karunya Transport Directorate.

8. RFC 7519. (2015). *JSON Web Token (JWT) Standards Specification*. Internet Engineering Task Force (IETF).